Assessment Tool 1 - Application-Based Questions

Student Name	POLI REDDY GOVARDHAN REDDY
Register No.	192311249
Course Name	Internet Programming
Course Code	CSA4303
Course Outcome Covered	CO-1 (L2): Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery.
GitHub Link	https://github.com/vinaykumar-bille/Internet_programming.git

Question 1: Online Symposium Portal

1. Missing HTML5 semantic elements

The given snippet is only a bare `<form>`. It has no document structure and no semantic landmarks, so browsers, search engines and screen readers cannot tell what each part of the page means. The following are missing:

- **Document-level:** `<!DOCTYPE html>`, `<html lang="en">`, `<head>` with `<meta charset>`, `<meta name="viewport">` and `<title>`.
- **Landmark elements:** `<header>` (portal title/logo), `<nav>` (site links), `<main>` (primary content), `<section>` (registration block with a heading) and `<footer>` (contact/copyright).
- **Form semantics:** `<label>` for every input, `<fieldset>` with `<legend>` to group related fields, and a heading (`<h1>` / `<h2>`) describing the form.
- **Attributes:** `id` on inputs (to bind labels), `required`, `autocomplete`, and `type="submit"` on the button.

2. Standards-compliant structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Symposium Registration</title>
</head>
<body>
  <header>
    <h1>National Technical Symposium</h1>
    <nav aria-label="Main navigation">
      <a href="/">Home</a>
      <a href="/events">Events</a>
      <a href="/contact">Contact</a>
    </nav>
  </header>
  <main>
    <section aria-labelledby="reg-heading">
      <h2 id="reg-heading">Student Registration</h2>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Participant details</legend>
          <label for="name">Full name</label>
          <input type="text" id="name" name="name" required
                 autocomplete="name">
          <label for="email">Email address</label>
          <input type="email" id="email" name="email" required
                 autocomplete="email">
        </fieldset>
        <button type="submit">Submit</button> </form>
    </section>
  </main>
  <footer>
    <p>&copy; 2026 College Symposium Committee</p> </footer>
</body>
</html>
```

This version has a valid doctype, declared language and encoding, correctly nested elements, and semantic landmarks. Every control has a programmatically associated label.

3. HTTP request generated on submission

Because `method="POST"`, the browser sends the field values in the request body, not the URL. The default encoding is `application/x-www-form-urlencoded`, with each name=value pair joined by `&` and special characters percent-encoded. For a student entering Arun Kumar and arun@example.com:

```
POST /register HTTP/1.1
Host: symposium.example.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 40
User-Agent: Mozilla/5.0
Accept: text/html
Cookie: sessionid=abc123

name=Arun+Kumar&email=arun%40example.com
```

- **Request line:** method (POST), target path (`/register`, resolved from `action`), protocol version.
- **Headers:** Host identifies the server; Content-Type tells the server how to parse the body; Content-Length gives the body size in bytes.

- **Body:** the encoded form data (`@` becomes `%40`, space becomes `+`).

POST is appropriate because registration creates or changes data on the server, so it is not a safe or idempotent operation.

4. HTTP response cycle from server to browser

1. **Connection:** the browser resolves the domain through DNS, opens a TCP connection (and a TLS handshake if HTTPS).
2. **Request sent:** the POST request above is transmitted.
3. **Server processing:** the web server passes the request to the application, which validates the data, stores the registration in a database and prepares a result.
4. **Response generated:** the server returns a status line, headers and an optional body, for example:

```
HTTP/1.1 302 Found
Location: /register/success
Set-Cookie: flash=registered; HttpOnly; Secure
```

or directly `HTTP/1.1 200 OK` with `Content-Type: text/html; charset=UTF-8` and an HTML confirmation page.

5. **Redirect (Post/Redirect/Get):** on a 302/303 the browser automatically issues `GET /register/success`, which returns 200 OK with the confirmation page. This stops duplicate registrations when the user refreshes.
6. **Rendering:** the browser parses the HTML into the DOM, requests sub-resources (CSS, JS, images), builds the render tree and displays the page.
7. **Connection reuse or close:** under HTTP/1.1 the connection is kept alive for further requests. Common status codes in this cycle: 200 OK (success), 302/303 (redirect), 400 Bad Request (invalid data), 409 Conflict (already registered), 500 Internal Server Error (server fault).

5. Improvements for usability and accessibility

Usability

- Use `required`, `type="email"`, `pattern` and `placeholder` hints for instant client-side validation, and always re-validate on the server.
- Show clear, specific error messages and keep the entered values after an error.
- Add `autocomplete` attributes so browsers can autofill.
- Give a visible confirmation page or message after submission.
- Use a responsive layout (viewport meta, flexible CSS) so the form works on mobile.
- Make the button text meaningful, for example "Register for Symposium". **Accessibility**
- Associate each `<label for>` with its input `id`; never rely on placeholder alone.
- Group fields with `<fieldset>` / `<legend>` and use landmark elements for screen reader navigation.
- Use ARIA only where native HTML is insufficient (`aria-describedby` for hints, `aria-live` for error announcements).
- Ensure full keyboard operation with a logical tab order and a visible focus outline.
- Maintain sufficient colour contrast (WCAG 2.1 AA, at least 4.5:1) and do not use colour alone to show errors.
- Add `lang` on `<html>` so assistive technology pronounces content correctly.

Question 2: Cross-Browser Compatibility Issue

Given snippet:

```
<html>
<head>
  <title>Company</title>
</head>
<body>
  <div>
    <h1>Welcome
    <p>About us
  </div>
</body>
</html>
```

1. Improper nesting and missing closing tags

- `<h1>Welcome` has no closing `</h1>`.
- `<p>About us` has no closing `</p>`.
- Because `<h1>` is never closed, the `<p>` is parsed as a child of the `<h1>`. A paragraph inside a heading is invalid nesting, since `<h1>` accepts only phrasing content.
- The `<div>`, `<body>` and `<html>` tags are closed correctly, but the `<div>` is forced to end the unclosed elements implicitly.
- Other omissions: no `<!DOCTYPE html>`, no `lang` attribute, no `<meta charset>`, and no viewport meta tag.

2. How different browsers may interpret this structure

- **No DOCTYPE triggers quirks mode.** Browsers then emulate legacy behaviour. Box-model calculation, default font sizes in tables, line-height and image alignment differ between browsers in quirks mode, which is the main source of the inconsistent layout.
- **Error recovery differs.** HTML5 defines a parsing algorithm, but older or non-conforming engines recover from unclosed tags in different ways. Some nest the `<p>` inside the `<h1>` (so it inherits heading size and weight), while others close the heading earlier, producing different spacing and font styling.
- **No charset declaration.** The browser may guess the encoding, so special characters can display wrongly.
- **No viewport meta tag.** Mobile browsers render at a virtual desktop width and scale down.
- **No reset or normalised CSS.** Each browser applies its own default margins and font sizes to `<h1>`, `<p>` and `<div>`.

3. Missing semantic elements

The page uses a generic `<div>` for everything. Missing are `<header>` (site banner/title), `<nav>` (menu), `<main>` (main content), `<section>` or `<article>` (the "About us" content), `<footer>` (contact and copyright) and optionally `<aside>`. Semantic tags improve accessibility, SEO and maintainability, and give browsers a predictable structure.

4. Corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
  <link rel="stylesheet" href="normalize.css">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Welcome</h1>
    <nav aria-label="Main navigation">
      <a href="/">Home</a>
      <a href="/about">About</a>
    </nav>
  </header>
  <main>
    <section>
      <h2>About Us</h2>
      <p>About us</p>
    </section>
  </main>
  <footer>
    <p>&copy; 2026 Company</p>
  </footer>
</body>
</html>
```

All elements are properly opened, nested and closed. The document is valid under the W3C Nu HTML Checker.

5. Techniques to ensure cross-browser compatibility

- Always start with `<!DOCTYPE html>` to force standards mode.
- Validate markup with the W3C Markup Validator and fix all errors.
- Use a CSS reset or normalize.css to remove default style differences.
- Use `box-sizing: border-box` consistently and modern layout (Flexbox/Grid) with fallbacks.
- Add vendor prefixes automatically using Autoprefixer, and check support on Can I Use or MDN compatibility tables.
- Use feature detection (`@supports`, Modernizr) instead of browser sniffing, and progressive enhancement with polyfills where needed.
- Use responsive design: viewport meta, relative units (`rem`, %) and media queries.
- Declare `<meta charset="UTF-8">`.
- Test on multiple browsers and devices (Chrome, Firefox, Safari, Edge) using real devices or tools such as BrowserStack and DevTools device emulation.

Question 3: Form Submission Failure

Given configuration:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

1. Issues in the form configuration

- `type="button"` **on the button.** It is a plain push button with no default action, so it does not submit the form. This is the root cause of the failure.
- `method="GET"` **for a login form.** The credentials would be appended to the URL (`/submit?username=...&password=...`), exposing the password in browser history, server logs, proxies and the Referer header.
- No `<label>` elements and no `id` attributes, which hurts accessibility.
- No validation attributes (`required`) and no autocomplete hints.
- No HTTPS or CSRF protection is indicated, and there is no surrounding valid document structure.

2. Why the HTTP request is not triggered

A form is submitted when the user activates a submit button (`<button type="submit">`, `<input type="submit">`) or presses Enter in a field with a submit button present. The default type of `<button>` inside a form is `submit`, but here it is explicitly set to `type="button"`. A button type fires only a click event and performs no built-in behaviour. Since no JavaScript handler calls `form.submit()` or `fetch()`, the browser never builds a request, and no HTTP request is sent to `/submit`. Clicking "Login" just does nothing.

3. Role of the GET method in this scenario

- GET is designed to retrieve resources. It is safe and idempotent, with parameters sent in the query string of the URL.
- If the form did submit, the browser would send: `GET /submit?username=admin&password=secret123 HTTP/1.1`.
- This leaves the password visible in the address bar, history, bookmarks, server and proxy logs, and the Referer header, and it can be cached. URL length limits also apply.
- GET is suitable for search or filtering forms, but not for login, which involves sensitive data and a state change (creating a session). POST should be used instead.

4. Corrected implementation

```
<form action="/submit" method="POST">
  <fieldset>
    <legend>Login</legend>
    <label for="username">Username</label>
    <input type="text" id="username" name="username" required
           autocomplete="username">
    <label for="password">Password</label>
    <input type="password" id="password" name="password" required
           minlength="8" autocomplete="current-password">
    <input type="hidden" name="csrf_token" value="{{ token }}">
    <button type="submit">Login</button>
  </fieldset>
</form>
```

Changes made: the method is POST, the button is `type="submit"`, labels and ids are added, validation and autocomplete attributes are included, and a CSRF token is added. Resulting request:

```
POST /submit HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
 username=admin&password=secret123&csrf_token=...
```

5. Improvements for secure data transmission

- Use HTTPS (TLS) for the page and the form action so credentials are encrypted in transit. Enable HSTS to prevent downgrade to HTTP.
- Use POST, never GET, for credentials.
- Hash passwords server-side with a slow algorithm (bcrypt, Argon2) with salt. Never store or log plain text passwords.
- Add CSRF tokens and set cookies with `Secure`, `HttpOnly` and `SameSite` flags.
- Perform server-side validation and sanitisation to prevent SQL injection and XSS. Use parameterised queries.
- Apply rate limiting, account lockout or CAPTCHA to stop brute-force attacks, and consider multi-factor authentication.
- Use a Content Security Policy and avoid exposing sensitive details in error messages (for example, show "Invalid username or password" only).
- Use `autocomplete="current-password"` so password managers work, and avoid putting sensitive data in URLs, cookies or local storage.

Code of Conduct

I, M. Yugandhar Reddy (Reg. No: 192372016), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Yugandhar Reddy.